Test Cases – E-Commerce API (DummyJSON)

Project: E-Commerce API Testing
Tool: Postman
API Under Test: Dummy Json {{DbaseURL}}
Test Types: Functional, Schema Validation, Business Logic, Negative Testing

---

Table of Contents

Products API – Valid Paths

TC-P-001 – Get All Products

Field	Details
Test Case ID	TC-P-001
Module	Products API
Test Type	Functional
Endpoint	`GET {{DbaseURL}}products`
Pre-condition	API is accessible, `DbaseURL` environment variable is set
Test Data	None

Steps:

1. Send a `GET` request to `{{DbaseURL}}products`
2. Observe the response status code
3. Parse and inspect the response body

Expected Results:

#	Assertion	Expected
1	Status code	200
2	Response has `products` property	Yes
3	`products` is a non-empty array	Yes
4	Each product has `id` (number), `title` (string), `price` (number > 0)	Yes
5	Each product rating is between 0 and 5	Yes
6	All product IDs are unique	Yes

TC-P-002 – Get Single Product

Field	Details
Test Case ID	TC-P-002
Module	Products API
Test Type	Functional
Endpoint	`GET {{DbaseURL}}products/1`
Pre-condition	Product with ID `1` exists
Test Data	Product ID: `1`

Steps:

1. Send a `GET` request to `{{DbaseURL}}products/1`

2. Observe the response status code
3. Parse and inspect the response body

Expected Results:

#	Assertion	Expected
1	Status code	200
2	`response.id` matches requested ID (`1`)	Yes
3	Response has `id` (number > 0), `title` (non-empty string), `price` (number > 0), `category` (non-empty string)	Yes
4	`rating` is a number between 0 and 5	Yes
5	`stock` is a number ≥ 0	Yes

TC-P-003 – Search Products with Pagination

Field	Details
Test Case ID	TC-P-003
Module	Products API
Test Type	Functional / Pagination Validation
Endpoint	`GET {{DbaseURL}}products?limit=10&skip=4`
Pre-condition	API is accessible,`DbaseURL` environment variable is set
Test Data	Query params: `limit=10`, `skip=4`

Steps:

1. Send a `GET` request to `{{DbaseURL}}products?limit=10&skip=4`
2. Observe the response status code
3. Validate pagination fields and product count

Expected Results:

#	Assertion	Expected
1	Status code	200
2	Response has `products` (array), `limit` (number), `skip` (number), `total` (number)	Yes
3	`products` array length equals 10	Yes
4	`skip` value in response equals 4	Yes
5	`products` count does not exceed `limit`	Yes

TC-P-004 – Create Product

Field	Details
Test Case ID	TC-P-004
Module	Products API
Test Type	Functional / Data Validation
Endpoint	`POST {{DbaseURL}}products/add`
Pre-condition	API is accessible,`DbaseURL` environment variable is set
Test Data	`{ "title": "Test Product", "price": 100 }`

Steps:

1. Send a `POST` request to `{{DbaseURL}}products/add` with the request body
2. Observe the response status code
3. Validate the response body against the request data

Expected Results:

#	Assertion	Expected
1	Status code	201
2	Response has `id` (number > 0), `title` (non-empty string), `price` (number > 0)	Yes
3	`response.title` matches request body `title`	Yes
4	`response.price` matches request body `price`	Yes
5	`price` is above 0	Yes

TC-P-005 – Register Token (Login)

Field	Details
Test Case ID	TC-P-005
Module	Auth / Products API
Test Type	Functional / Auth Validation
Endpoint	`POST {{DbaseURL}}/auth/login`
Pre-condition	Valid user credentials exist
Test Data	`{ "username": "emilys", "password": "emilyspass" }`

Steps:

1. Send a `POST` request to `{{DbaseURL}}/auth/login` with credentials
2. Observe the response status code
3. Validate tokens and user details
4. Confirm token is saved to collection variable

Expected Results:

#	Assertion	Expected

1	Status code	200
2	`accessToken` exists and is a non-empty string	Yes
3	`refreshToken` exists and is a non-empty string	Yes
4	`id` is a number > 0	Yes
5	`username` is a non-empty string	Yes
6	`email` contains `@`	Yes
7	`firstName` and `lastName` are non-empty strings	Yes
8	Response `username` matches request body `username`	Yes
9	`accessToken` is saved to collection variable	Yes

TC-P-006 – Get All Products Categories

Field	Details
Test Case ID	TC-P-006
Module	Products API
Test Type	Functional
Endpoint	`GET {{DbaseURL}}products/categories`
Pre-condition	API is accessible
Test Data	None

Steps:

1. Send a `GET` request to `{{DbaseURL}}products/categories`
2. Observe the response status code
3. Inspect each category object in the response

Expected Results:

#	Assertion	Expected
1	Status code	200
2	Response is a non-empty array	Yes
3	Each item has `slug` (non-empty string)	Yes
4	Each item has `name` (non-empty string)	Yes
5	Each item has `url` that includes `https://`	Yes

TC-P-007 – Get All Product Category List

Field	Details
Test Case ID	TC-P-007
Module	Products API
Test Type	Functional / Data Integrity
Endpoint	`GET{{DbaseURL}}products/category-list`
Pre-condition	API is accessible
Test Data	None

Steps:

1. Send a `GET` request to `{{DbaseURL}}products/category-list`
2. Observe the response status code
3. Validate array contents and uniqueness

Expected Results:

#	Assertion	Expected
1	Status code	200

2	Response is a non-empty array	Yes
3	Every item is a non-empty string	Yes
4	All category names are unique	Yes

TC-P-008 – Update Product (PATCH)

Field	Details
Test Case ID	TC-P-008
Module	Products API
Test Type	Functional / Data Validation
Endpoint	`PATCH {{DbaseURL}}products/1`
Pre-condition	Product with ID exists
Test Data	Updated product fields in request body

Steps:

1. Send a `PATCH` request to `{{DbaseURL}}products/1` with updated fields
2. Observe the response status code
3. Validate the returned product structure and updated data

Expected Results:

#	Assertion	Expected
1	Status code	200
2	`response.id` matches URL product ID	Yes
3	Response has `id`, `title`, `price`, `discountPercentage`, `stock`, `rating`, `description`, `brand`, `category`	Yes
4	All fields have correct data types	Yes
5	Price,Stock-Updated field value matches the request body value	Yes

---

TC-P-009 – Delete Product

Field	Details
Test Case ID	TC-P-009
Module	Products API
Test Type	Functional
Endpoint	`DELETE {{DbaseURL}}products/3`
Pre-condition	Product with ID exists
Test Data	Product ID:

Steps:

1. Send a `DELETE` request to `{{DbaseURL}}products/3`
2. Observe the response status code
3. Validate the deletion confirmation in the response body

Expected Results:

#	Assertion	Expected
1	Status code	200
2	Response contains `id` field	Yes
3	Deleted product `id` matches requested ID (`3`)	Yes
4	`isDeleted` is `true`	Yes
5	`deletedOn` is not null	Yes

Products API – Invalid Paths

TC-P-010 – Login with Invalid Credentials

Field	Details
Test Case ID	TC-P-010
Module	Auth / Products API
Test Type	Negative Testing
Endpoint	`POST {{DbaseURL}}/auth/login`
Pre-condition	API is accessible
Test Data	`{ "username": "wqeafsdgn", "password": "dsv" }`

Steps:

1. Send a `POST` request with invalid username and password
2. Observe the response status code
3. Validate the error message

Expected Results:

#	Assertion	Expected
1	Status code	400
2	Response has `message` (non-empty string)	Yes
3	`message` equals `"Invalid credentials"`	Yes

TC-P-011 – Login Without Credentials

Field	Details
Test Case ID	TC-P-011
Module	Auth / Products API
Test Type	Negative Testing
Endpoint	`POST {{DbaseURL}}/auth/login`
Pre-condition	API is accessible
Test Data	Empty request body

Steps:

1. Send a `POST` request with an empty body
2. Observe the response status code
3. Validate the error message

Expected Results:

#	Assertion	Expected
1	Status code	400
2	Response has `message` (non-empty string)	Yes
3	`message` equals `"Username and password required"`	Yes

TC-P-012 – Get Product with Invalid ID

Field	Details
Test Case ID	TC-P-012
Module	Products API
Test Type	Negative Testing

Endpoint	`GET {{DbaseURL}}/products/195`
Pre-condition	Product with ID `195` does not exist
Test Data	Product ID: `195`

Steps:

1. Send a `GET` request with a non-existent product ID (`195`)
2. Observe the response status code
3. Validate the error message content

Expected Results:

#	Assertion	Expected
1	Status code	404
2	Response has `message` (non-empty string)	Yes
3	`message` equals `"Product with id '195' not found"`	Yes
4	`message` contains `"195"`	Yes

TC-P-013 – Search Products with Invalid Skip Value

Field	Details
Test Case ID	TC-P-013
Module	Products API
Test Type	Negative Testing / Boundary
Endpoint	`GET {{DbaseURL}}products?skip=abc`
Pre-condition	API is accessible

Test Data	Query param: `skip=abc`

Steps:

1. Send a `GET` request with a non-numeric `skip` value
2. Observe the response status code
3. Validate the error message

Expected Results:

#	Assertion	Expected
1	Status code	400
2	Response has `message` (non-empty string)	Yes
3	`message` equals `"Invalid 'skip' - must be a number"`	Yes

TC-P-014 – Search Products with Invalid Limit Value

Field	Details
Test Case ID	TC-P-014
Module	Products API
Test Type	Negative Testing / Boundary
Endpoint	`GET {{DbaseURL}}products?skip=10&limit=-`
Pre-condition	API is accessible
Test Data	Query params: `skip=10`, `limit=-`

Steps:

1. Send a `GET` request with a non-numeric `limit` value (`-`)
2. Observe the response status code
3. Validate the error message

Expected Results:

#	Assertion	Expected
1	Status code	400
2	Response has `message` (non-empty string)	Yes
3	`message` equals `"Invalid 'limit' - must be a number"`	Yes

Carts API – Valid Paths

TC-C-001 – Get All Carts

Field	Details
Test Case ID	TC-C-001
Module	Carts API
Test Type	Functional / Schema Validation
Endpoint	`GET {{DbaseURL}}carts`
Pre-condition	API is accessible
Test Data	None

Steps:

1. Send a `GET` request to `{{DbaseURL}}carts`
2. Observe the response status code
3. Validate the response structure and cart fields

Expected Results:

#	Assertion	Expected

1	Status code	200
2	Response has `carts` (array)	Yes
3	`carts` array has at least one item(product)	Yes
4	Each cart has expected fields with correct data types	Yes

TC-C-002 – Get Single Cart

Field	Details
Test Case ID	TC-C-002
Module	Carts API
Test Type	Functional / Business Logic Validation
Endpoint	`GET {{DbaseURL}}carts/1`
Pre-condition	Cart with ID `1` exists
Test Data	Cart ID: `1`

Steps:

1. Send a `GET` request to `{{DbaseURL}}carts/1`
2. Observe the response status code
3. Validate cart structure, product fields, and all business logic calculations

Expected Results:

#	Assertion	Expected
1	Status code	200
2	`response.id` matches requested cart ID	Yes
3	Cart has `id`, `products`, `total`, `discountedTotal`, `totalProducts`, `totalQuantity`	Yes

4	Each product has `id`, `title`, `price`, `quantity`, `total`, `discountPercentage`, `discountedTotal`, `thumbnail`	Yes
5	All product fields have correct data types	Yes
6	Cart `total` = sum of all product `total` values (±0.01)	Yes
7	Product `total` = `price × quantity` (±0.01)	Yes
8	Cart `discountedTotal` = sum of product `discountedTotal` values (±0.01)	Yes
9	`totalQuantity` = sum of all product `quantity` values	Yes
10	`discountedTotal` is less than `total`	Yes
11	`totalProducts` = length of products array	Yes

TC-C-003 – Get Cart by User

Field	Details
Test Case ID	TC-C-003
Module	Carts API
Test Type	Functional / Data Integrity
Endpoint	`GET {{DbaseURL}}carts/user/5`
Pre-condition	User with ID `5` has carts
Test Data	User ID: `5`

Steps:

1. Send a `GET` request to `{{DbaseURL}}carts/user/5`
2. Observe the response status code
3. Validate all returned carts belong to the requested user

Expected Results:

#	Assertion	Expected
1	Status code	200
2	Response has `carts` (non-empty array)	Yes
3	Every cart's `userId` matches the requested user ID (`5`)	Yes
4	Cart array is not empty	Yes
5	Products array inside each cart is not empty	Yes

TC-C-004 – Add a New Cart

Field	Details
Test Case ID	TC-C-004
Module	Carts API
Test Type	Functional / Data Validation
Endpoint	`POST {{DbaseURL}}carts/add`
Pre-condition	Valid user ID and product IDs exist
Test Data	`{ "userId": 1, "products": [{ "id": 144, "quantity": 4 }, { "id": 22, "quantity": 1 }] }`

Steps:

1. Send a `POST` request with a valid `userId` and products array
2. Observe the response status code
3. Validate the created cart response

Expected Results:

#	Assertion	Expected
1	Status code	201
2	Response body has valid CartId which is a number	Yes
3	Response body has valid `userId` matching request body	Yes
4	Each cart has expected fields (`id`, `total`, `discountedTotal`, `totalProducts`, `totalQuantity`)	Yes
5	Response body has valid `products` array (non-empty)	Yes
6	Each product has required fields (`id`, `title`, `price`, `quantity`, `total`)	

TC-C-005 – Update Cart (PATCH)

Field	Details
Test Case ID	TC-C-005
Module	Carts API
Test Type	Functional / Data Validation
Endpoint	`PATCH {{DbaseURL}}carts/1`
Pre-condition	Cart with ID `1` exists
Test Data	Updated products in request body

Steps:

1. Send a `PATCH` request to `{{DbaseURL}}carts/1` with updated data
2. Observe the response status code
3. Validate the cart ID and updated product data

Expected Results:

#	Assertion	Expected
1	Status code	200
2	`response.id` matches the request URL cart ID	Yes
3	Updated product data reflects the changes sent in the request	Yes

---

TC-C-006 – Delete Cart

Field	Details
Test Case ID	TC-C-006
Module	Carts API
Test Type	Functional
Endpoint	`DELETE {{DbaseURL}}carts/1`
Pre-condition	Cart with ID `1` exists
Test Data	Cart ID: `1`

Steps:

1. Send a `DELETE` request to `{{DbaseURL}}carts/1`
2. Observe the response status code
3. Validate the deletion response

Expected Results:

#	Assertion	Expected
1	Status code	200
2	Response contains `id` field	Yes

3	Deleted product `id` matches requested ID (`3`)	Yes
4	`isDeleted` is `true`	Yes
5	`deletedOn` is not null	Yes

Carts API – Invalid Paths

TC-C-007 – Get Cart with Invalid ID

Field	Details
Test Case ID	TC-C-007
Module	Carts API
Test Type	Negative Testing
Endpoint	`GET {{DbaseURL}}carts/12345`
Pre-condition	Cart with ID `12345` does not exist
Test Data	Cart ID: `12345`

Steps:

1. Send a `GET` request with a non-existent cart ID (`12345`)
2. Observe the response status code
3. Validate the error message contains the requested cart ID

Expected Results:

#	Assertion	Expected
1	Status code	404

2	Response has `message` (non-empty string)	Yes
3	`message` contains the cart ID `12345`	Yes

TC-C-008 – Add a New Cart with Invalid User ID

Field	Details
Test Case ID	TC-C-008
Module	Carts API
Test Type	Negative Testing
Endpoint	`POST {{DbaseURL}}carts/add`
Pre-condition	API is accessible
Test Data	`{ "userId": -1, "products": [{ "id": 144, "quantity": 4 }, { "id": 22, "quantity": 1 }] }`

Steps:

1. Send a `POST` request with an invalid `userId` of `-1`
2. Observe the response status code
3. Validate the error message contains the invalid userId

Expected Results:

#	Assertion	Expected
1	Status code	404
2	Response has `message` (non-empty string)	Yes
3	`message` contains the invalid `userId` (`-1`)	Yes

TC-C-009 – Add a New Cart with No Data

Field	Details
Test Case ID	TC-C-009
Module	Carts API
Test Type	Negative Testing
Endpoint	`POST {{DbaseURL}}carts/add`
Pre-condition	API is accessible
Test Data	Empty request body

Steps:

1. Send a `POST` request with an empty body
2. Observe the response status code
3. Validate the error message

Expected Results:

#	Assertion	Expected
1	Status code	400
2	Response has `message` (non-empty string)	Yes
3	`message` equals `"User id is required"`	Yes

Summary

Module	Total TCs	Valid Path	Invalid Path
Products API	14	9	5
Carts API	9	6	3
Total	**23**	**15**	**8**